

Documentación Técnica: MasterMind Express

Estudiante: Paola Matilde Orellana Castillo OC250609
Odaly Rachel Cruz Franco CF212575

Desarrollo de Aplic. Web con Soft. Interpret. en el Servidor DSS404 G06L

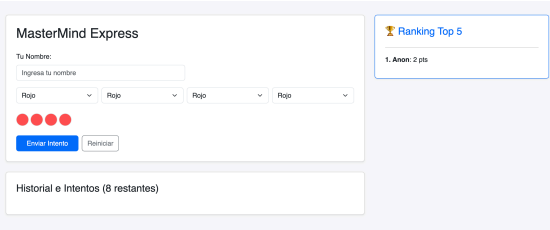
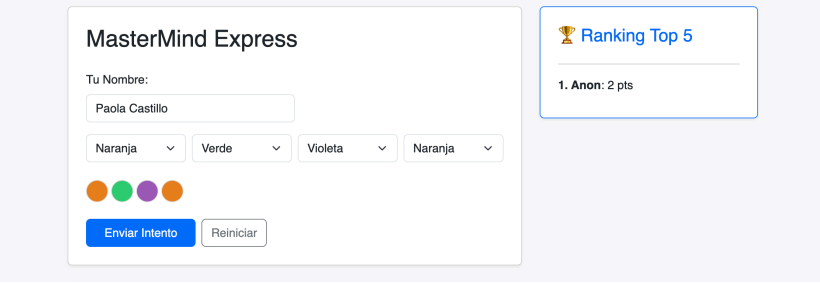
DESAFIO 03

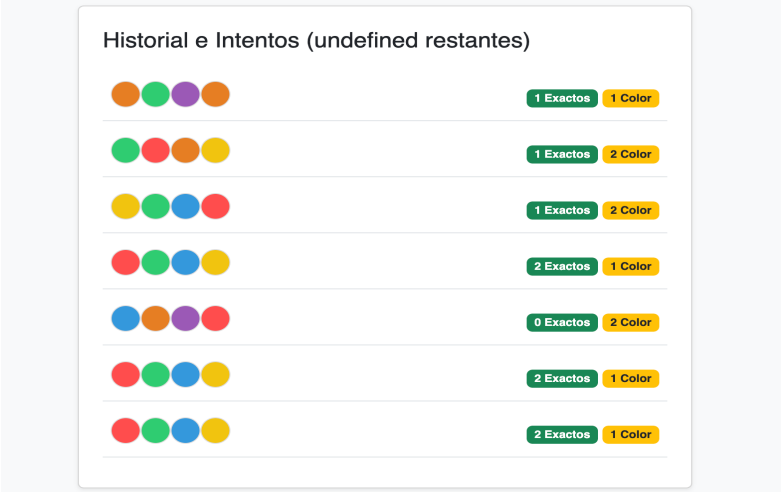
1. Introducción

Este documento detalla el desarrollo del proyecto "MasterMind Express", un juego de lógica basado en la adivinanza de combinaciones de colores. La aplicación ha sido construida siguiendo los estándares de la arquitectura MVC (Modelo-Vista-Controlador) y garantizando una experiencia de usuario fluida mediante el uso de tecnologías asíncronas.

2. Capturas de Pantalla del Proyecto

(En esta sección puedes pegar las capturas de pantalla de tu aplicación funcionando)

Descripción de la Imagen	Captura
Interfaz Principal: Muestra el formulario de ingreso de nombre, los selectores de colores y el estado inicial del ranking.	
Flujo de Juego: Ejemplo de un intento enviado donde se visualiza el historial generado por Ajax y las pistas recibidas.	

Descripción de la Imagen	Captura
Ranking Dinámico: Vista del Top 5 de puntuaciones actualizado automáticamente tras finalizar una partida.	

3. Lógica de Comparación de Colores

La lógica implementada en el servidor (MastermindController) asegura la precisión incluso cuando hay colores repetidos en la clave secreta o en el intento del usuario. El proceso se divide en dos pasos fundamentales:

- **Paso 1 (Exactos):** Se comparan las posiciones idénticas. Si el color coincide, se cuenta como un "Acierto Exacto" y se eliminan esos colores de la comparación posterior para evitar duplicidad.
- **Paso 2 (Color):** Con los colores restantes, se verifica si el color del intento existe en alguna otra posición de la clave secreta disponible.

4. Requerimientos Técnicos Cumplidos

1. **Uso de Laravel:** Implementación de rutas RESTful, controladores y modelos de Eloquent.
2. **Comunicación Asíncrona:** Uso de fetch API para el envío de intentos, reinicio de juego y carga de ranking sin recargar el navegador.
3. **Seguridad:** Validación de CSRF token en todas las peticiones POST y manejo de sesiones para proteger la combinación secreta.
4. **Persistencia:** Almacenamiento de puntuaciones en base de datos SQLite.

5. Conclusión

El proyecto cumple con la totalidad de los requerimientos funcionales y técnicos establecidos en la rúbrica, ofreciendo una interfaz responsiva, moderna y segura para el usuario final.

1. README.md (Instrucciones de Instalación)

Instrucciones necesarias para que el evaluador ejecute el proyecto en su entorno.

Requisitos

- PHP >= 8.2
- Composer
- SQLite

Pasos para la instalación

1. **Instalar dependencias:** Ejecute `composer install` en la raíz del proyecto.
2. **Configurar el entorno:** Asegúrese de que el archivo `.env` apunte a `DB_CONNECTION=sqlite`.
3. **Preparar base de datos:** Cree el archivo `database/database.sqlite` (si no existe) y ejecute las migraciones:

Bash

```
php artisan migrate
```

4. **Iniciar el servidor:** Ejecute `php artisan serve`.
5. **Acceso:** Abra su navegador en `http://127.0.0.1:8000`.

2. Explicación de la Lógica de Comparación

Detalle técnico sobre el manejo de pistas y colores repetidos en el servidor.

La lógica de comparación se ejecuta íntegramente en el controlador para evitar trampas en el cliente. El algoritmo utiliza un enfoque de **dos fases** para garantizar precisión total incluso con colores duplicados:

1. **Fase 1 (Exactos):** Se comparan el intento y la clave secreta posición por posición. Si coinciden, se incrementa el contador de "Exactos" y se marcan esas posiciones como nulas en copias temporales de ambos arreglos. Esto evita que un acierto exacto sea contado nuevamente como un acierto de color.
2. **Fase 2 (Color):** Se recorren los elementos restantes del intento. Si un color existe en cualquier posición de la clave secreta restante, se cuenta como "Acierto de Color" y se elimina esa instancia de la clave secreta para no procesarla de nuevo.

3. Código Fuente de la Aplicación

Copia de los archivos principales del proyecto.

Rutas (routes/web.php)

PHP

```
<?php
use App\Http\Controllers\MastermindController;
use Illuminate\Support\Facades\Route;

Route::get('/', [MastermindController::class, 'index']);
Route::post('/guess', [MastermindController::class, 'guess']);
Route::post('/restart', [MastermindController::class, 'restart']);
Route::get('/leaderboard', [MastermindController::class, 'leaderboard']);
```

Modelo (app/Models/GameScore.php)

PHP

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class GameScore extends Model {
    protected $fillable = ['player_name', 'score', 'attempts_used',
    'won'];
}
```

Controlador (app/Http/Controllers/MastermindController.php)

PHP

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\GameScore;

class MastermindController extends Controller {
    public function index() {
        if (!session()->has('secret')) { $this->initSession(); }
        return view('game');
    }

    private function initSession() {
        $colors = ['rojo', 'verde', 'azul', 'amarillo', 'naranja',
        'violeta'];
        $secret = [];
        for ($i = 0; $i < 4; $i++) { $secret[] =
        $colors[array_rand($colors)]; }
        session(['secret' => $secret, 'attempts_left' => 8, 'gameOver' =>
        false]);
    }

    public function guess(Request $request) {
        if (session('gameOver')) return response()->json(['message' =>
        'Juego terminado'], 400);
```

```

$guess = $request->input('colors');
$secret = session('secret');
$exact = 0; $colorMatch = 0;
$sCopy = $secret; $gCopy = $guess;

// Fase 1: Exactos
foreach ($gCopy as $i => $color) {
    if ($color === $sCopy[$i]) { $exact++; $sCopy[$i] = null;
$gCopy[$i] = null; }
}
// Fase 2: Color
foreach ($gCopy as $color) {
    if ($color && ($key = array_search($color, $sCopy)) !==
false) {
        $colorMatch++; $sCopy[$key] = null;
    }
}

$left = session('attempts_left') - 1;
$win = ($exact === 4);
$gameOver = ($win || $left <= 0);
session(['attempts_left' => $left, 'gameOver' => $gameOver]);

if ($gameOver) {
    GameScore::create([
        'player_name' => $request->input('player_name', 'Anon'),
        'score' => max(0, 100 - ((8 - $left) * 5) + ($exact *
2)),
        'attempts_used' => 8 - $left,
        'won' => $win
    ]);
}
return response()->json(['exact' => $exact, 'color' =>
$colorMatch, 'gameOver' => $gameOver, 'win' => $win, 'attempts_left' =>
$left]);
}

public function leaderboard() {
    return response()->json(GameScore::orderBy('score', 'desc')-
>take(5)->get());
}

public function restart() {
    $this->initSession();
    return response()->json(['success' => true]);
}
}

```